

Tree-2

Question No:1

CS	Session	Tree 2	Question Description	Level 1	Challenge 7 / 1
Question description Given two rooted trees, your task is to find out if they are <i>isomorphic</i> , i.e., it is possible to draw them so that they look the same. Constraints • $1 \leq t \leq 1000$ • $2 \leq n \leq 10^5$ • the sum of all values of n is at most 10^5 Input The first input line has an integer t : the number of tests. Then, there are t tests described as follows: The first line has an integer n : the number of nodes in both trees. The nodes are numbered $1, 2, \dots, n$, and node 1 is the root. Then, there are $n-1$ lines describing the edges of the first tree, and finally $n-1$ lines describing the edges of the second tree. Output For each test, print "YES", if the trees are isomorphic, and "NO" otherwise.					

Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 2 3 1 2 2 3 1 2 1 3 3 1 2 2 3 1 3 3 2	INPUT (STDIN) 2 3 1 2 2 3 1 3 3 3 3 1 2 2 3 1 3 3 3
EXPECTED OUTPUT NO YES	EXPECTED OUTPUT NO NO

Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #define MAXN 100005
6  #define HASH_SIZE 200003
7
8  typedef struct Edge {
9      int to;
10     int next;
11 } Edge;
12
13 Edge edges1[2 * MAXN], edges2[2 * MAXN];
14 int head1[MAXN], head2[MAXN];
15 int edgeCount1, edgeCount2;
16
17 /* Store canonical signatures */
18 typedef struct Signature {
19     int *arr;
20     int len;
21     int id;
22     struct Signature *next;
23 } Signature;
24
25 Signature *table[HASH_SIZE];
26 int nextID;
27
```

```

28  /* ----- Utility ----- */
29
30  void addEdge(Edge edges[], int *edgeCount, int head[],
31             int u, int v)
32  {
33      edges[*edgeCount].to = v;
34      edges[*edgeCount].next = head[u];
35      head[u] = (*edgeCount)++;
36  }
37
38  /* Comparator for sorting integers */
39  int compareInt(const void *a, const void *b)
40  {
41      return (*(int *)a - *(int *)b);
42  }
43
44  /* Simple hash for a vector */
45  unsigned long long hashVector(int *arr, int len)
46  {
47      unsigned long long h = 1469598103934665603ULL;
48
49      for (int i = 0; i < len; i++)
50      {
51          h ^= (unsigned long long)(arr[i] + 1000003);
52          h *= 1099511628211ULL;
53      }

```

```
55     h ^= len;
56     return h;
57 }
58
59 /* Get canonical ID for a sorted vector */
60 int getID(int *arr, int len)
61 {
62     unsigned long long h = hashVector(arr, len);
63     int bucket = h % HASH_SIZE;
64
65     Signature *cur = table[bucket];
66
67     while (cur != NULL)
68     {
69         if (cur->len == len &&
70             memcmp(cur->arr, arr, len * sizeof(int)) == 0)
71         {
72             return cur->id;
73         }
74
75         cur = cur->next;
76     }
77
78     /* New structure */
79     Signature *node = (Signature *)malloc(sizeof(Signature));
```

```

81     node->arr = NULL;
82
83     if (len > 0)
84     {
85         node->arr = (int *)malloc(len * sizeof(int));
86         memcpy(node->arr, arr, len * sizeof(int));
87     }
88
89     node->len = len;
90     node->id = nextID;
91
92     node->next = table[bucket];
93     table[bucket] = node;
94
95     nextID++;
96
97     return node->id;
98 }
99
100 /* ----- Free hash table ----- */
101
102 void clearTable()
103 {
104     for (int i = 0; i < HASH_SIZE; i++)
105     {
106         Signature *cur = table[i];
107

```

```
108         while (cur != NULL)
109         {
110             Signature *temp = cur;
111             cur = cur->next;
112
113             free(temp->arr);
114             free(temp);
115         }
116
117         table[i] = NULL;
118     }
119
120     nextID = 1;
121 }
122
123 /* ----- Find canonical ID ----- */
124
125 int getTreeID(Edge edges[], int head[], int n)
126 {
127     int *parent = (int *)calloc(n + 1, sizeof(int));
128     int *order = (int *)malloc((n + 1) * sizeof(int));
129     int *stack = (int *)malloc((n + 1) * sizeof(int));
130
131     int top = 0;
132     int orderSize = 0;
```

```
134      /* Root is 1 */
135      stack[top++] = 1;
136      parent[1] = -1;
137
138      /* Iterative DFS */
139      while (top > 0)
140      {
141          int u = stack[--top];
142
143          order[orderSize++] = u;
144
145          for (int e = head[u]; e != -1; e = edges[e].next)
146          {
147              int v = edges[e].to;
148
149              if (v == parent[u])
150                  continue;
151
152              parent[v] = u;
153              stack[top++] = v;
154          }
155      }
156
157      int *nodeID = (int *)calloc(n + 1, sizeof(int));
158
159      /* Bottom-up processing */
```

```
160     for (int i = orderSize - 1; i >= 0; i--)
161     {
162         int u = order[i];
163
164         int childCount = 0;
165
166         /* Count children */
167         for (int e = head[u]; e != -1; e = edges[e].next)
168         {
169             int v = edges[e].to;
170
171             if (parent[v] == u)
172                 childCount++;
173         }
174
175         int *childIDs = NULL;
176
177         if (childCount > 0)
178         {
179             childIDs = (int *)malloc(childCount * sizeof(int));
180
181             int index = 0;
182
183             for (int e = head[u]; e != -1; e = edges[e].next)
184             {
185                 int v = edges[e].to;
```

```
186
187         if (parent[v] == u)
188         {
189             childIDs[index++] = nodeID[v];
190         }
191     }
192
193     /*
194     * Child order does not matter,
195     * so sort the child IDs.
196     */
197     qsort(childIDs, childCount,
198           sizeof(int), compareInt);
199 }
200
201 /*
202 * The sorted list of child IDs uniquely
203 * represents the structure of this subtree.
204 */
205 nodeID[u] = getID(childIDs, childCount);
206
207 free(childIDs);
208 }
```

```

210     int result = nodeID[1];
211
212     free(parent);
213     free(order);
214     free(stack);
215     free(nodeID);
216
217     return result;
218 }
219
220 /* ----- Main ----- */
221
222 int main()
223 {
224     int T;
225     scanf("%d", &T);
226
227     while (T--)
228     {
229         int n;
230         scanf("%d", &n);
231
232         /* Initialize adjacency lists */
233         for (int i = 1; i <= n; i++)
234         {
235             head1[i] = -1;

```

```

236         head2[i] = -1;
237     }
238
239     edgeCount1 = 0;
240     edgeCount2 = 0;
241
242     /* First tree */
243     for (int i = 0; i < n - 1; i++)
244     {
245         int u, v;
246         scanf("%d %d", &u, &v);
247
248         addEdge(edges1, &edgeCount1, head1, u, v);
249         addEdge(edges1, &edgeCount1, head1, v, u);
250     }
251
252     /* Second tree */
253     for (int i = 0; i < n - 1; i++)
254     {
255         int u, v;
256         scanf("%d %d", &u, &v);
257
258         addEdge(edges2, &edgeCount2, head2, u, v);
259         addEdge(edges2, &edgeCount2, head2, v, u);
260     }
261
262     /*
263      * Both trees must use the SAME canonical-ID table.
264      * Therefore structurally identical subtrees receive
265      * the same ID.
266      */
267     clearTable();
268
269     int id1 = getTreeID(edges1, head1, n);
270     int id2 = getTreeID(edges2, head2, n);
271
272     if (id1 == id2)
273         printf("YES\n");
274     else
275         printf("NO\n");
276
277     clearTable();
278 }
279
280 return 0;
281 }

```

Output:

Test Case-1:

	2	
	3	
	1 2	
	2 3	
	1 2	
	1 3	
	NO	
	3	
	1 2	
	2 3	
	1 3	
	3 2	
	YES	

Test Case-2:

	2	
	3	
	1 2	
	2 3	
	1 3	
	3 3	
	NO	
	3	
	1 2	
	2 3	
	1 3	
	3	
	3	
	NO	

Keshav Kumar Rauniyar

CH.SC.U4CSE25236